

Fundamentos de Sistemas Robóticos com ROS 2

Etapa 3

Professor Mario Pego

mario.pegó@prof.infnet.edu.br



Cronograma



Fundamentos de Sistemas Robóticos com ROS 2

<i>Etapas</i>	<i>Segunda-feira/Quarta-feira</i>
Etapa 1	20/07/26 e 22/07/26
Etapa 2/Leitura TP1	27/07/26 e 29/07/26
Etapa 3	03/08/26 e 05/08/26
Etapa 4/Entrega do TP1/Leitura TP2	10/08/26 e 12/08/2026
Etapa 5	17/08/26 e 19/08/26
Etapa 6/Entrega TP2/Leitura do TP3	24/08/26 e 26/08/26
Etapa 7	31/08/26 e 02/09/26
Etapa 8/Entrega TP3	07/09/26 (FERIADO) e 09/09/26
Etapa 9/Leitura do AT	14/09/26 e 16/09/26
Entrega Assessment	21/09/26 e 23/09/26
Reentrega	28/09/26 e 30/09/26

Explorar os conceitos centrais
de comunicação e descoberta
do ecossistema ROS 2

O que vamos ver

1

O grafo computacional do ROS 2

Nós, tópicos, serviços e ações como vértices e arestas

2

Tópicos, serviços e ações

Como funcionam e quando usar cada um

3

DDS e descoberta automática

O middleware que conecta os nós sem um servidor central

4

ROS_DOMAIN_ID e namespaces

Isolamento lógico de sistemas robóticos

5

CLI de inspeção

ros2 node, ros2 topic e ros2 service na prática

O grafo computacional do ROS 2

Nós, tópicos, serviços e ações são os vértices e arestas de um grafo distribuído

- Nó (node): processo independente que executa uma tarefa específica (ler um sensor, planejar uma trajetória, publicar comandos)
- Cada nó roda isoladamente e pode estar em máquinas diferentes
- Os nós se conectam entre si através de tópicos, serviços e ações
- O conjunto de nós + conexões ativas em um instante é chamado de grafo computacional (computation graph)

Por que isso importa?

Todo comando de inspeção do ROS 2 (ros2 node, ros2 topic, ros2 service) é, na prática, uma consulta a esse grafo em tempo real. Entender essa metáfora é a base para depurar qualquer sistema robótico distribuído.

Um robô como grafo de nós

Exemplo simplificado de um robô móvel

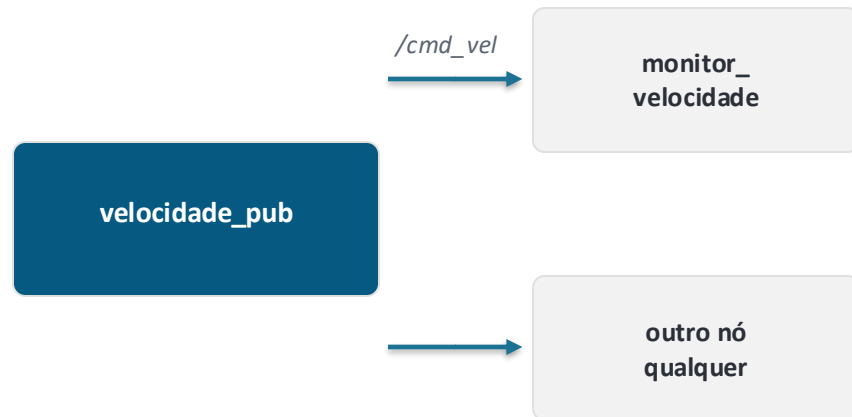


Cada seta é uma conexão via tópico — os nós não se conhecem diretamente, só o nome e o tipo do canal.

Tópicos: comunicação assíncrona

Publish/Subscribe — o mecanismo mais usado no ROS 2

- Um publisher envia mensagens para um canal nomeado (o tópico), sem saber quem vai recebê-las
- Um ou mais subscribers recebem cada mensagem publicada, de forma assíncrona
- Relação muitos-para-muitos: vários publishers e vários subscribers podem coexistir no mesmo tópico
- No TP2, /cmd_vel vai ser exatamente esse canal: velocidade_pub publica, monitor_velocidade assina



O publisher não sabe quem (nem quantos) está ouvindo — é como um anúncio de rádio.

Mensagens: os pacotes de dados

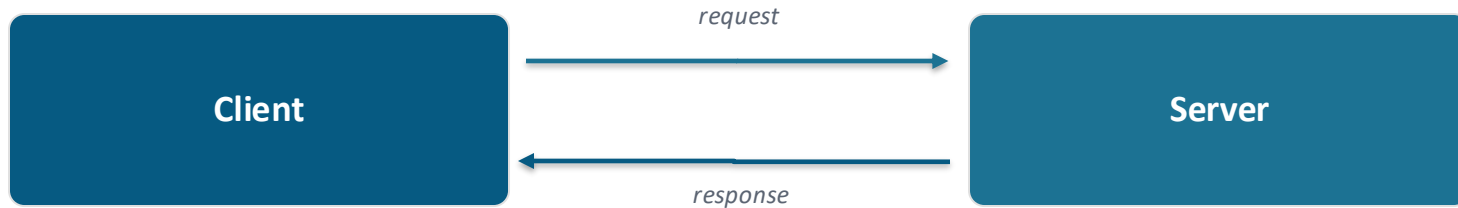
```
# Pacotes de interface mais comuns
std_msgs/msg/String
std_msgs/msg/Int32
geometry_msgs/msg/Twist
geometry_msgs/msg/Pose
sensor_msgs/msg/Image
sensor_msgs/msg/LaserScan
```

`interfaces .msg`

Cada tipo de mensagem é definido em um arquivo `.msg` com campos tipados. O TP2 usa `geometry_msgs/msg/Twist` no `/cmd_vel` — é isso que você importa com `'from geometry_msgs.msg import Twist'`.

Serviços: comunicação síncrona

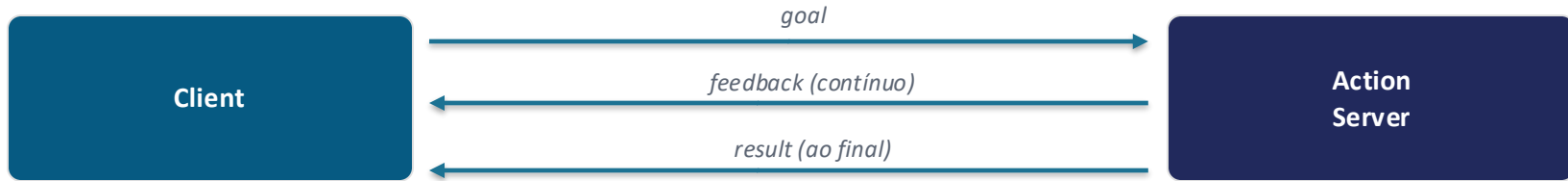
Request/Response — um-para-um



No TP2, modo_client (client) envia True/False para modo_server (server), que responde com uma mensagem de confirmação — exatamente esse padrão request/response.

Ações: tarefas longas com feedback

Para operações que não terminam instantaneamente



- Combina o melhor de tópico e serviço: envia um objetivo (*goal*) e recebe feedback contínuo até o resultado final
- Suporta cancelamento durante a execução
- Exemplo típico: navegar até uma coordenada, informando % de progresso a cada segundo

Tópicos vs. Serviços vs. Ações

Critério	Tópico	Serviço	Ação
Sincronismo	Assíncrono	Síncrono	Assíncrono c/ feedback
Partes envolvidas	N publishers : N subscribers	1 client : 1 server	1 client : 1 action server
Feedback intermediário	Não	Não	Sim, contínuo
Pode ser cancelado	Não se aplica	Não	Sim
Caso de uso típico	Streaming de sensor, cmd_vel	Ligar/desligar um modo	Navegar até um ponto

Escolhendo o mecanismo certo

- "Preciso transmitir dados continuamente e não importa se alguém perde uma mensagem?" → Tópico (ex.: /cmd_vel do velocidade_pub)
- "Preciso de uma confirmação imediata de que uma ação pontual foi executada?" → Serviço (ex.: modo_server ligando/desligando um modo)
- "A tarefa demora, preciso acompanhar o progresso e talvez cancelar?" → Ação (ex.: mover um braço robótico até uma posição)
- **Regra prática: comece sempre pelo tópico; use serviço quando precisar de confirmação síncrona; use ação só quando o feedback contínuo for essencial.**

DDS: o middleware por trás do ROS 2

Data Distribution Service — o protocolo real de comunicação

- O ROS 2 não implementa rede do zero: ele delega toda comunicação a uma implementação DDS (ex.: Fast DDS, Cyclone DDS)
- DDS é um padrão da indústria (OMG) usado também em aviação, defesa e telecom
- Fornece descoberta automática, QoS configurável e comunicação peer-to-peer, sem um broker central
- rclpy e rclcpp são camadas que traduzem a API do ROS 2 para chamadas DDS

Por que isso importa?

Diferente do ROS 1 (que dependia de um roscore central), no ROS 2 não existe ponto único de falha: se um nó cair, os demais continuam se comunicando normalmente.

Descoberta automática

Como os nós se encontram na rede sem um servidor central

- Cada nó, ao iniciar, anuncia sua presença via multicast UDP na rede local
- Os demais nós no mesmo domínio recebem esse anúncio e trocam metadados (tópicos publicados/assinados, tipos, QoS)
- Esse processo é contínuo: um nó que entra depois é descoberto automaticamente, sem reiniciar os outros
- Não existe um 'roscore' — a descoberta é totalmente distribuída

ROS_DOMAIN_ID: isolamento lógico

O experimento que o TP2 pede no Exercício 4

```
# Terminal do velocidade_pub
$ export ROS_DOMAIN_ID=42
$ ros2 run comunicacao_ros2 \
    velocidade_pub

# Terminal do monitor_velocidade
# (sem definir ROS_DOMAIN_ID -> domínio 0)
$ ros2 run comunicacao_ros2 \
    monitor_velocidade
```

Resultado esperado

Com domínios diferentes (42 vs. 0), o monitor_velocidade não recebe nenhuma mensagem — é exatamente essa falha que o TP2 pede para você demonstrar e depois corrigir.

Namespaces: organizar por robô ou subsistema

O namespace /robo1 pedido no TP2

```
$ ros2 run comunicacao_ros2 \
  velocidade_pub \
  --ros-args -r __ns:=/robo1

# resultado:
# nó:      /robo1/velocidade_pub
# tópico:  /robo1/cmd_vel
```

No TP2

O Exercício 4 pede para modificar o `velocidade_pub.py` para publicar em `/robo1/cmd_vel` — o namespace é aplicado via `--ros-args -r __ns:=/robo1`, sem alterar a lógica do nó.

ROS_DOMAIN_ID vs. Namespace

Duas formas de isolamento — não são a mesma coisa

Critério	ROS_DOMAIN_ID	Namespace
Nível	Rede (DDS)	Nomenclatura (grafo)
Efeito	Nós não se descobrem entre si	Nós se descobrem, mas com nomes prefixados
Uso típico	Isolar equipes/robôs na mesma rede física	Organizar múltiplas instâncias do mesmo robô
No TP2	ROS_DOMAIN_ID=42 no velocidade_pub	namespace /robo1 no /cmd_vel

Demonstrando o isolamento por domínio

Dois terminais em ROS_DOMAIN_ID diferentes

Terminal 1 — velocidade_pub

```
$ export ROS_DOMAIN_ID=42
$ ros2 run comunicacao_ros2 \
  velocidade_pub --ros-args \
  -r __ns:=/robo1
```

Terminal 2 — monitor_velocidade

```
$ ros2 run comunicacao_ros2 \
  monitor_velocidade

(nenhuma saída — sem mensagens)
```

Diagnóstico

O Terminal 2 não define ROS_DOMAIN_ID (fica no domínio padrão 0), enquanto o Terminal 1 está no domínio 42. Os nós nunca se descobrem — o monitor_velocidade fica esperando mensagens que nunca chegam.

Corrigindo o isolamento

Alinhando o ROS_DOMAIN_ID nos dois terminais

Terminal 1 — velocidade_pub (sem alteração)

```
$ export ROS_DOMAIN_ID=42
$ ros2 run comunicacao_ros2 \
  velocidade_pub --ros-args \
  -r __ns:=/robo1
```

Terminal 2 — monitor_velocidade (corrigido)

```
$ export ROS_DOMAIN_ID=42
$ ros2 run comunicacao_ros2 \
  monitor_velocidade

[INFO] v=0.50 m/s, sentido=anti-horário
```

Resultado

Com o mesmo ROS_DOMAIN_ID nos dois terminais, a descoberta automática volta a funcionar e o monitor_velocidade passa a receber as mensagens publicadas em /robo1/cmd_vel — é essa evidência (antes/depois) que o TP2 pede em prints de terminal.

Inspecionando nós

ros2 node list e ros2 node info — comandos pedidos no TP2 Exercício 4

```
$ ros2 node list
/velocidade_pub
/monitor_velocidade

$ ros2 node info /velocidade_pub
Publishers: /cmd_vel [geometry_msgs/msg/Twist]
Subscribers: (nenhum)
```

Leitura

ros2 node info mostra exatamente o que aquele nó publica, assina, e quais serviços/ações oferece — o primeiro comando a rodar ao depurar um grafo desconhecido.

Inspecionando tópicos

ros2 topic list, info e hz — validação do velocidade_pub no TP2

```
$ ros2 topic list -t
/cmd_vel [geometry_msgs/msg/Twist]

$ ros2 topic hz /cmd_vel
average rate: 2.001

$ ros2 topic echo /cmd_vel
```

Leitura

hz mede a frequência real de publicação — o TP2 pede que ela fique próxima de 2.0 Hz; echo imprime as mensagens Twist conforme chegam, em tempo real.

Os 5 comandos de validação

Exatamente os comandos que o TP2 pede para colar (com interpretação) no relatório

Comando	O que revela
<code>ros2 node list</code>	Todos os nós ativos no grafo no momento
<code>ros2 node info /velocidade_pub</code>	Publishers, subscribers, serviços e ações daquele nó
<code>ros2 topic list -t</code>	Todos os tópicos ativos com seus tipos de mensagem
<code>ros2 topic hz /cmd_vel</code>	Frequência real de publicação (esperado: ~2.0 Hz)
<code>ros2 topic info /cmd_vel --verbose</code>	Tipo, publishers/subscribers conectados e QoS de cada um

Inspecionando serviços

ros2 service list e type — validação do modo_server no TP2

```
$ ros2 service list  
/velocidade_pub/describe_parameters  
/modo
```

```
$ ros2 service type /modo  
std_srvs/srv/SetBool
```

Leitura

Todo nó expõe automaticamente serviços internos de introspecção (parâmetros, logging) — o serviço 'modo' que você mesmo criar aparece junto na lista.

Visualizando o grafo com rqt_graph

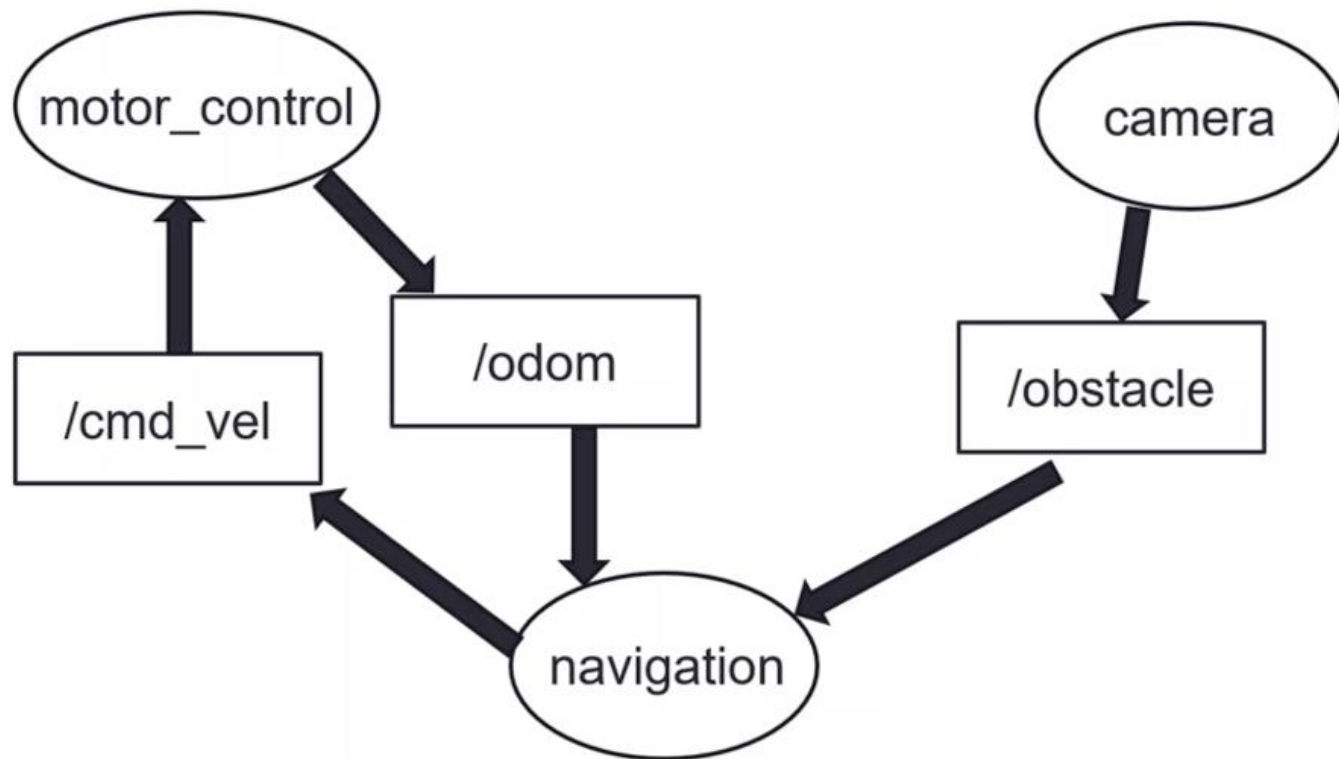
Uma alternativa gráfica à inspeção via terminal

- rqt_graph desenha nós como bolhas e tópicos/serviços como setas entre elas, em tempo real
- Útil para conferir visualmente se velocidade_pub e monitor_velocidade estão realmente conectados pelo /cmd_vel
- Comando: `ros2 run rqt_graph rqt_graph`
- Complementa (não substitui) a CLI — para detalhes de tipo e QoS, a CLI ainda é necessária

Dica

Em sistemas com dezenas de nós, comece pelo rqt_graph para ter a visão geral, depois use `ros2 topic info --verbose` nos tópicos específicos que precisar depurar.

Exemplo de Aplicação



Nomenclatura e organização do grafo

- Use nomes descritivos e em snake_case: /cmd_vel, /battery_state, não /t1 ou /dados
- Prefira namespaces a nomes compostos manualmente (/robo1/cmd_vel em vez de /robo1_cmd_vel)
- Documente o tipo de cada tópico/serviço customizado no README do pacote
- Evite nomes genéricos demais (/data, /status) que colidem entre pacotes diferentes
- Um nó, uma responsabilidade: velocidade_pub só publica, monitor_velocidade só assina

Por que usar `snake_case`?



✓ Mais legível

Fácil de ler e entender, mesmo em projetos grandes.



✓ É o padrão da linguagem Python (PEP 8)

É o padrão oficial definido no PEP 8.



✓ É a convenção adotada na maioria dos pacotes do ROS e ROS 2

Pacotes, nós, tópicos e serviços seguem `snake_case`.



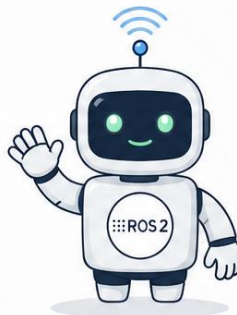
✓ Facilita a manutenção e a colaboração

Padrão consistente ajuda toda a equipe a trabalhar junta com menos erros.

Exemplos no ROS 2

✓ Correto (`snake_case`)

	Pacote:	<code>robot_navigation</code>
	Nó:	<code>laser_scanner</code>
	Tópico:	<code>/scan</code>
	Serviço:	<code>/reset_odometry</code>
	Variável:	<code>robot_speed</code>
	Parâmetro:	<code>max_speed</code>



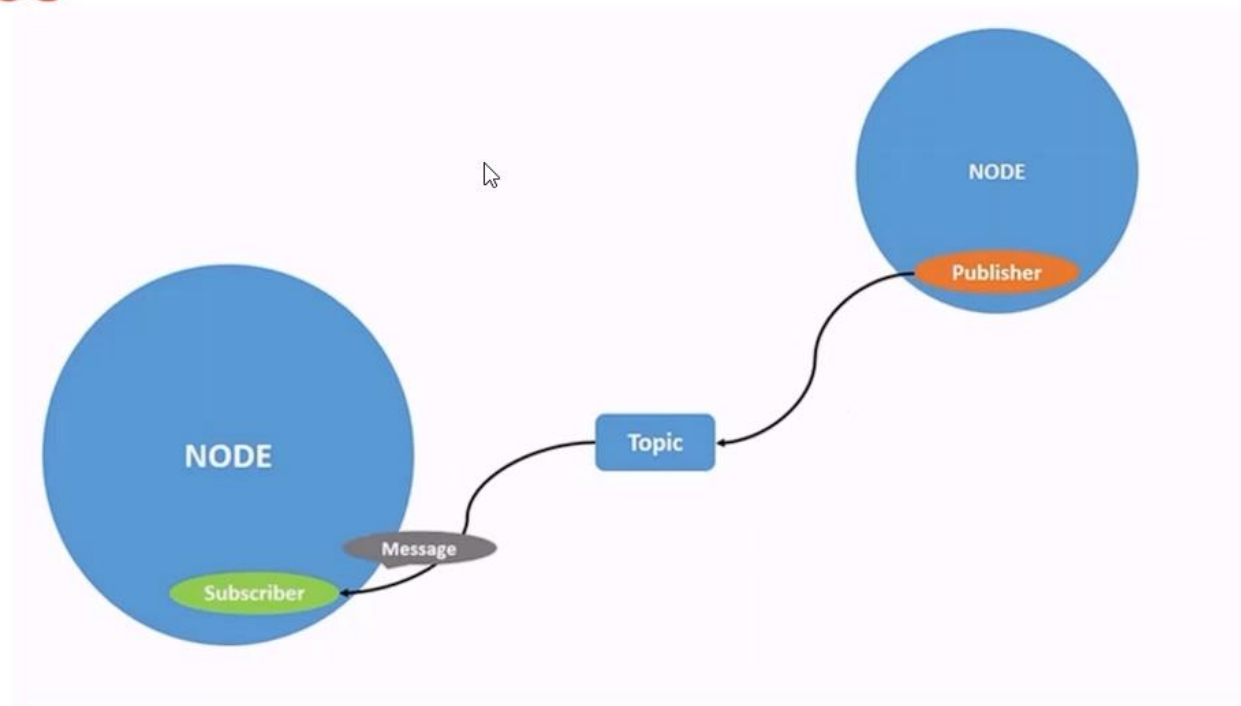
✗ Evite (não recomendado)

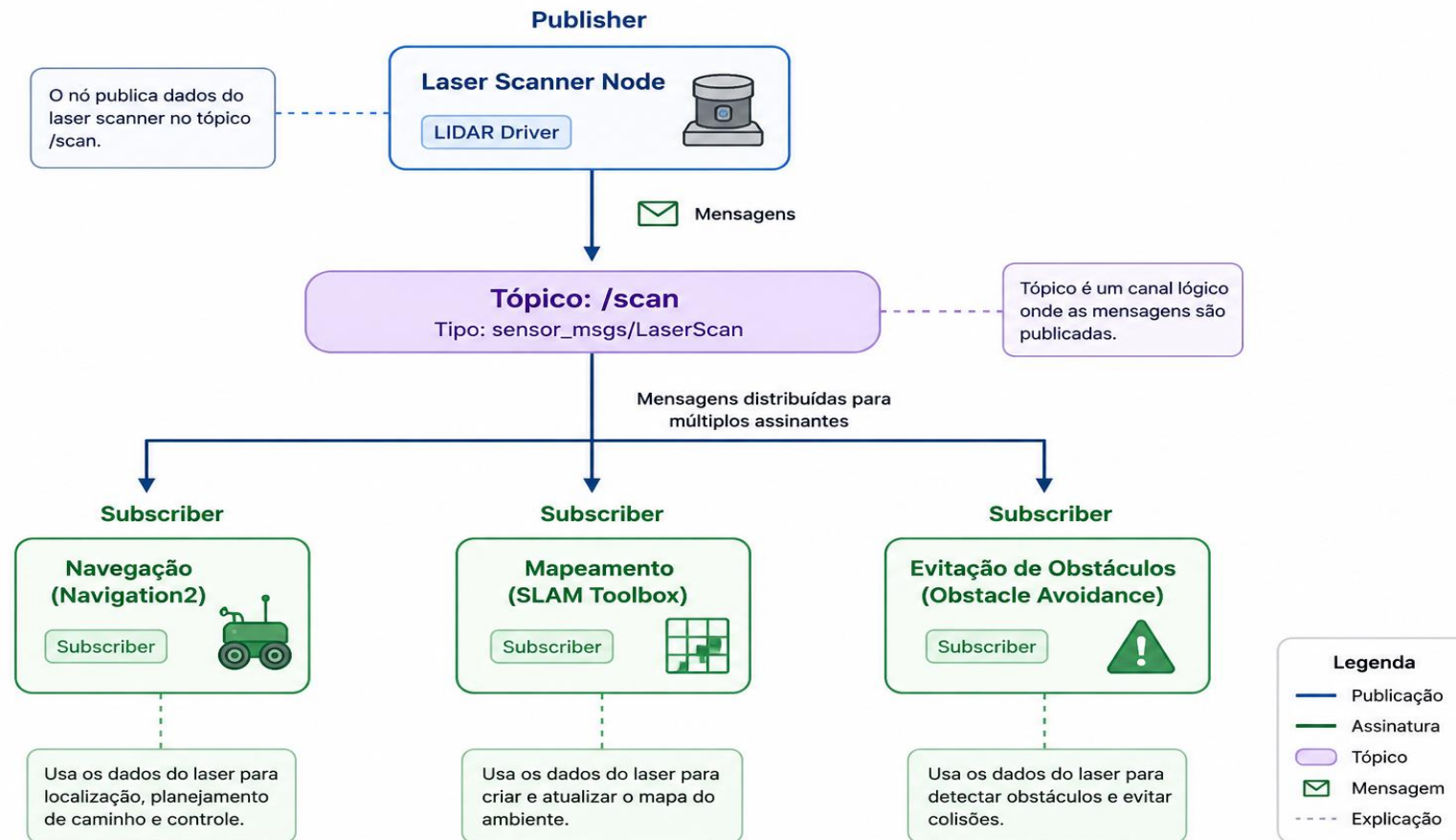
	Pacote:	<code>RobotNavigation</code>
	Nó:	<code>LaserScannerNode</code>
	Tópico:	<code>/Scan</code>
	Serviço:	<code>/ResetOdometry</code>
	Variável:	<code>robotSpeed</code>
	Parâmetro:	<code>MaxSpeed</code>



Dica: Use `snake_case` sempre que possível em projetos ROS 2 com Python. Isso torna seu código mais profissional e fácil de manter!

Tópicos





Competência 1

OK

Tópico / Serviço / Ação

diferenciar e escolher o mecanismo certo

OK

`ros2 node list / info`

inspecionar nós ativos no grafo

OK

`ros2 topic list -t / hz / --
verbose`

consigo rodar os 5 comandos de validação do Exercício 4

OK

`ROS_DOMAIN_ID=42 / namespace
/robo1`

demonstrar a falha e a correção do isolamento (Exercício 4)

Próximo passo

Agora que entendemos como os nós se comunicam e se descobrem, vamos colocar a mão no código: na próxima competência, você vai criar o pacote `comunicacao_ros2` e executar seus primeiros nós Python com o build system Colcon — a base para o `velocidade_pub`, `monitor_velocidade` e `modo_server` do TP2.

Criar e executar nós ROS 2 em Python utilizando o build system Colcon

O que vamos ver

1

Anatomia de um pacote Python

package.xml, setup.py, setup.cfg

2

Nó mínimo em rclpy

Ciclo de vida, logging e timers

3

Build com Colcon

colcon build, entry points e overlay

4

Executando e depurando

ros2 run e erros comuns

Relembrando o tp1_ws

O que já fizemos no TP1

```
$ mkdir -p ~/tp1_ws/src  
$ cd ~/tp1_ws  
$ colcon build
```

```
tp1_ws/  
├── build/  install/  log/  
└── src/    (ainda vazio)
```

Onde paramos

No TP1 criamos o workspace e validamos o build vazio. Agora vamos criar dentro de src/ o pacote comunicacao_ros2, que o TP2 pede — com os quatro executáveis que vamos construir nesta e na próxima etapa.

Anatomia de um pacote Python ROS 2

Estrutura mínima gerada pelo `ros2 pkg create`

```
comunicacao_ros2/  
├─ package.xml      # metadados e dependências  
├─ setup.py         # build + entry points (executáveis)  
├─ setup.cfg        # opções do instalador Python  
├─ resource/comunicacao_ros2  
├─ comunicacao_ros2/ # código-fonte (módulo Python)  
│   ├── __init__.py  
│   ├── velocidade_pub.py  
│   ├── monitor_velocidade.py  
│   ├── modo_server.py  
│   └── modo_client.py  
└─ test/            # testes automatizados (opcional)
```

package.xml

Como o arquivo deve ficar depois de editado à mão

Salvará em `comunicacao_ros2/package.xml`

```
<package format="3">
  <name>comunicacao_ros2</name>
  <version>0.0.1</version>
  <description>Nós de comunicação — TP2</description>
  <maintainer email="a@b.com">Aluno</maintainer>
  <license>Apache-2.0</license>

  <depend>rclpy</depend>
  <depend>geometry_msgs</depend>
  <depend>std_srvs</depend>
</package>
```

Atenção: isto NÃO é o que o comando gera

O 'ros2 pkg create' preenche version, description, maintainer e license com TODO (ex.: 'TODO: Package description') — é normal e não é erro. As <depend> já vêm certas por causa do --dependencies. Edite os campos TODO à mão para chegar neste resultado antes de entregar o TP2.

setup.py

Onde os quatro executáveis do TP2 são registrados

Salvará em `comunicacao_ros2/setup.py`

```
entry_points={
    'console_scripts': [
        'velocidade_pub = '
        'comunicacao_ros2.velocidade_pub:main',
        'monitor_velocidade = '
        'comunicacao_ros2.monitor_velocidade:main',
        'modo_server = '
        'comunicacao_ros2.modo_server:main',
        'modo_client = '
        'comunicacao_ros2.modo_client:main',
    ],
},
```

entry_points

Cada linha cria um executável que o 'ros2 run comunicacao_ros2 <nome>' consegue encontrar. Os quatro nomes aqui já são exatamente os pedidos pelo TP2.

setup.cfg

Opções do instalador (gerado automaticamente)

Salvará em `comunicacao_ros2/setup.cfg`

```
[develop]
script_dir=$base/lib/comunicacao_ros2
[install]
install_scripts=$base/lib/comunicacao_ros2
```

Na prática

Esse arquivo é criado automaticamente pelo 'ros2 pkg create' e raramente precisa ser editado à mão — ele só garante que os scripts sejam instalados no diretório que o ROS 2 espera.

Criando o pacote comunicacao_ros2

```
ros2 pkg create --build-type ament_python
```

```
$ cd ~/tp1_ws/src
$ ros2 pkg create --build-type ament_python comunicacao_ros2 --dependencies rclpy geometry_msgs std_srvs

going to create a new package
package name: comunicacao_ros2
creating comunicacao_ros2/setup.py
```

--dependencies

Preenche automaticamente as tags <depend> no package.xml — inclua rclpy, geometry_msgs e std_srvs de uma vez, já que o pacote vai hospedar todos os exercícios do TP2.

Conferindo com tree

A estrutura criada dentro de src/

```
$ tree ~/tp1_ws/src/comunicacao_ros2
```

```
comunicacao_ros2/  
├── comunicacao_ros2/  
│   └── __init__.py  
├── package.xml  
├── resource/comunicacao_ros2  
├── setup.cfg  
├── setup.py  
└── test/
```

Nó mínimo em rclpy

O esqueleto de velocidade_pub.py, antes de adicionar o Publisher

```
comunicacao_ros2/comunicacao_ros2/velocidade_pub.py
```

```
import rclpy
from rclpy.node import Node

class VelocidadePub(Node):
    def __init__(self):
        super().__init__(
            'velocidade_pub')
        self.get_logger().info(
            'Nó iniciado')
```

Node

`super().__init__('velocidade_pub')` registra o nó no grafo computacional com esse nome — é ele que vai aparecer em `'ros2 node list'` no TP2. Na Etapa 4 adicionamos o `create_publisher`. Este fragmento sozinho ainda NÃO roda: falta o `main()` (próximo slide).

Ciclo de vida de um nó

init → spin → shutdown

```
comunicacao_ros2/comunicacao_ros2/velocidade_pub.py
```

```
def main(args=None) :  
    rclpy.init(args=args)  
    no = VelocidadePub()  
    rclpy.spin(no)      # bloqueia aqui  
    no.destroy_node()  
    rclpy.shutdown()  
  
if __name__ == '__main__':  
    main()
```

spin()

`rclpy.spin()` entra em um loop infinito que processa callbacks (timers, mensagens recebidas) até o processo ser interrompido (Ctrl+C) — é isso que mantém o nó 'vivo' quando você rodar 'ros2 run' no TP2. Este `main()` se junta à classe do slide anterior — o arquivo consolidado (com timer) está no próximo slide.

Logging com `get_logger()`

Nunca use `print()` em um nó ROS 2

```
self.get_logger().info('mensagem normal')  
self.get_logger().warn('algo suspeito')  
self.get_logger().error('falha grave')  
self.get_logger().debug('detalhe técnico')
```

Por quê?

Os logs do `get_logger()` incluem automaticamente timestamp, nome do nó e nível de severidade — e podem ser filtrados via `'ros2 run --log-level'`. `print()` não aparece nesses filtros nem nos arquivos de log.

Timers: executando código periodicamente

A base do `velocidade_pub.py` da Etapa 4

```
class VelocidadePub(Node):  
    def __init__(self):  
        super().__init__(  
            'velocidade_pub')  
        self.create_timer(  
            0.5,          # 2 Hz  
            self.tick)  
  
    def tick(self):  
        self.get_logger().info('tick')
```

Preview

0.5 segundos = 2 Hz, exatamente a frequência que o TP2 pede para o `velocidade_pub`. Na Etapa 4, o `tick()` vai publicar um Twist em vez de só logar.

velocidade_pub.py — versão esqueleto

Nó com timer a 2 Hz, ainda sem publicar (o Publisher vem na Etapa 4)

salvar em `comunicacao_ros2/comunicacao_ros2/velocidade_pub.py`

```
import rclpy
from rclpy.node import Node

class VelocidadePub(Node):
    def __init__(self):
        super().__init__('velocidade_pub')
        self.create_timer(0.5, self.tick)    # 2 Hz

    def tick(self):
        self.get_logger().info('publicando em /cmd_vel...')

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(VelocidadePub())
    rclpy.shutdown()
```

Registrando o executável

setup.py precisa apontar para a função main

salvar em `comunicacao_ros2/setup.py`

```
entry_points={
    'console_scripts': [
        'velocidade_pub = '
        'comunicacao_ros2.velocidade_pub:main',
    ],
},
```

Formato

'nome_do_comando =
pacote.arquivo:funcao' — é esse
'nome_do_comando' que você digita depois
em 'ros2 run comunicacao_ros2
velocidade_pub', exatamente como o TP2
vai validar.

Compilando com Colcon

colcon build --packages-select

```
$ cd ~/tp1_ws  
$ colcon build --packages-select comunicacao_ros2
```

```
Starting >>> comunicacao_ros2  
Finished <<< comunicacao_ros2 [1.2s]  
Summary: 1 package finished [1.4s]
```

--packages-select

Recompila só o comunicacao_ros2, em vez do workspace inteiro — muito mais rápido a cada novo executável (velocidade_pub, monitor_velocidade, modo_server, modo_client) que você adicionar.

Overlay vs. underlay

Por que é preciso dar 'source' depois de cada build

- Underlay: a instalação base do ROS 2 (/opt/ros/humble/setup.bash)
- Overlay: o seu workspace (~/.local/share/colcon/install/setup.bash), que 'empilha' sobre o underlay
- Sem dar source no overlay, o shell não enxerga o pacote comunicacao_ros2 que você acabou de compilar
- Cada terminal novo precisa repetir os dois source (ou automatizar via .bashrc)

Comando

```
source /opt/ros/humble/setup.bash  
source ~/.local/share/colcon/install/setup.bash
```

A ordem importa: o overlay deve ser sourced depois do underlay.

Executando o nó

ros2 run <pacote> <executável>

```
$ source ~/tp1_ws/install/setup.bash
$ ros2 run comunicacao_ros2 velocidade_pub

[INFO] [velocidade_pub]: publicando em /cmd_vel...
[INFO] [velocidade_pub]: publicando em /cmd_vel...
```

Confirmação

Se aparecer 'command not found' ou o pacote não for encontrado, o problema quase sempre é: esqueceu o source, ou o entry point não está registrado no setup.py.

Erros comuns e como resolver

Sintoma	Causa provável	Solução
Package not found	Overlay não sourced	source install/setup.bash
No executable found	Entry point ausente/errado	Conferir setup.py + rebuild
ModuleNotFoundError	Falta rebuild após editar	colcon build novamente
ros2: command not found	ROS 2 não sourced	source /opt/ros/humble/setup.bash

Organização do código

- Um arquivo por nó, nomeado igual ao nó (`velocidade_pub.py` → nó '`velocidade_pub`')
- Sempre registre o entry point no `setup.py` antes de tentar '`ros2 run`'
- Rode '`colcon build --packages-select`' após CADA alteração, mesmo pequena
- Mantenha o `get_logger()` como única forma de saída — nunca misture com `print()`
- Commite o `package.xml` e `setup.py` junto com o código — são parte do pacote

Inspecionando pacotes instalados

ros2 pkg list e executables

```
$ ros2 pkg list | grep comunicacao  
comunicacao_ros2
```

```
$ ros2 pkg executables comunicacao_ros2  
comunicacao_ros2 velocidade_pub
```

Quando usar

Útil para confirmar que o build realmente registrou o executável antes de tentar 'ros2 run' — evita ficar adivinhando se o problema é de build ou de digitação.

Competência 2

OK `ros2 pkg create` criar o pacote comunicacao_ros2 com as dependências certas

OK `setup.py entry_points` registrar os quatro executáveis do TP2

OK `colcon build --packages-select` compilar incrementalmente

OK `ros2 run` executar e depurar erros comuns

Próximo passo

Com o pacote `comunicacao_ros2` criado e o esqueleto do `velocidade_pub` rodando, a Etapa 4 vai completar os quatro executáveis: `velocidade_pub` e `monitor_velocidade` (Publishers/Subscribers com QoS), e `modo_server/modo_client` (serviços Client/Server) — (Exercícios 1, 2 e 3 do TP2).